太原理工大学 Intelligent Transportation

大学生智能交通创新实践基地

第十二届智能汽车竞赛暨 RoboMaster 机甲大师校内赛

结构赛题示例代码配套文档

V1.0.0 2025 10.1

V2.0.0 2026 8.23

太原理工大学

大学生智能交通创新实践基地

一、示例程序整体介绍

1.这些是函数声明，宏定义以及全局变量定义。（不要轻易修改原有的）

函数声明：只声明函数的名字、参数，提前告诉编译器函数存在，因为函数一般都是分装在后面，没有声明的话会报错。自己封装函数写在这里。（非必须封装）

```
253 }
254
255 /* USER CODE BEGIN 4 */
256
257
258
259 /* USER CODE END 4 */
260
261 /**
```

宏定义：给一段文本起个名字。这里给 1500，50 取名字，在之后使用 Center_Right_UD 时相当于使用 1500，eg. Right_UD = Center_Right_UD 就等于 Right_UD = 1500。

变量：存放数据的空间，在使用前需要先定义，不然编译器不认识你的变量，最好初始化赋值为 0，eg. Int a = 0; 不然空间内可能是乱数据。

```
91
92  /* Private function prototypes -----------------
93  void SystemClock_Config(void);
94  /* USER CODE BEGIN PFP */
95  void Prevent_sensitity(void);
96  void Prevent_Lost(void);
97  void Getswitch(void);
98  /* USER CODE END PFP */
99
00  /* Private user code ---------------------------
01  /* USER CODE BEGIN 0 */
02
03  #define Center_Right_UD 1500
04  #define Center_Right_LR 1500
05  #define Center_Left_UD  1500
06  #define Center_Left_LR  1500
07
08  #define Dead_zone 50
09
10  int Right_UD = 0;
11  int Right_LR = 0;
12  int Left_UD = 0;
13  int Left_LR = 0;
14
15  int Right_UD_PWM = 0;
16  int Right_LR_PWM = 0;
17  int Left_UD_PWM = 0;
18  int Left_LR_PWM = 0;
19
20  int RCSend_Lost = 0;
21  int vlf,vlb,vrf,vrb;//
22  int GiPwmLF,GiPwmRF,GiPwmLR,GiPwmRR;
```

2.这些是硬件初始化部分（芯片上电之后，把硬件、外设从 “乱的未知状态” 设置成 “我们想要的工作状态”，这个过程就叫初始化。）（不要修改 ）

```
121  int main(void)
122  {
123
124    /* USER CODE BEGIN 1 */
125
126    /* USER CODE END 1 */
127
128    /* MCU Configuration--------------------------------------
129
130    /* Reset of all peripherals, Initializes the Flash inte
131    HAL_Init();
132
133    /* USER CODE BEGIN Init */
134
135    /* USER CODE END Init */
136
137    /* Configure the system clock */
138    SystemClock_Config();
139
140    /* USER CODE BEGIN SysInit */
141
142    /* USER CODE END SysInit */
143
144    /* Initialize all configured peripherals */
145    MX_GPIO_Init();
146    MX_DMA_Init();
147    MX_USART1_UART_Init();
148    MX_USART3_UART_Init();
149    MX_TIM3_Init();
150    MX_TIM5_Init();
151    MX_TIM4_Init();
152    MX_TIM6_Init();
153    MX_TIM2_Init();
154    MX_ADC1_Init();
155    /* USER CODE BEGIN 2 */
156    PWM_Init();    // PWM
157    RC_Init();     // SBUS
158    Chassis_Init();    //
159
160    ADC_Init();        // start ADC+DMA, sample 6 channels
```

3.这个函数获得拨码开关的信号（高低电平），并且根据信号控制 led 灯，从而可以直观了解到拨码开关的情况。（不许修改）（tips.可以通过拨码开关的信号设置多套方案来适应场上的情况）点击一下这个函数会变蓝（如图），然后点击 F12 会跳转到该函数定义位置，了解具体内容。

```
Getswitch();//获得拨码开关的键值
Getswitch();//获得拨码开关的键值
```

4.这里是给遥控器传回来的值一个死区，也就是这在个变化范围内不进行处理，这是因为传回来的值会有小幅度波动，这种情况下就算摇杆在居中位置，电机也会有微小动作。

```
Prevent_sensitity();//防止过于遥控信号敏感
```

可以通过修改这个 Dead_zone 来修改敏感程度

```
#define Dead_zone 50
```

5.这里就是你大展身手的空间了！

```
218
219  /*============================================== User Code Space ==============================================*/
220
221  /*============================================== User Code   End ==============================================*/
222
```

6.这是对底盘四个电机的控制，你需要在上面计算出直行的参数和左右移动的参数（一个摇杆在不同位置的值是不一样的，用当下的摇杆值减去居中位置的摇杆值就可以知道摇杆的拨动情况，只要把一个遥杆前后的值减去中间值赋值给 hCtrl.Chassis.SpeedTarget_Y，左右的值减去中间的值赋值给 hCtrl.Chassis.SpeedTarget_X，在电机接线正确且其正反转正确的情况下，你就可以让小车移动了！（这个只是例子，大家还需要去测试具体合适的大小是多少，通过给系数来实现缩放，比如说 hCtrl.Chassis.SpeedTarget_Y 算出来只有 500，不足以驱动轮子转动 那么我就需要在使用他的时

候×6 来放大)

```
    hCtrl.Chassis.SpeedTarget_Y = Right_UD - Center_Right_UD;
    hCtrl.Chassis.SpeedTarget_X = Right_LR - Center_Right_LR;
  /*=================================================== User Code End ====================
    // 底盘驱动
    ChassisCtrl(hCtrl.Chassis.SpeedTarget_Y, hCtrl.Chassis.SpeedTarget_X);
```

二、如何把遥控器与小车上的接收机连接起来

连接遥控器和接收机起始很简单，不需要理解他背后的原理，只要按以下步骤作即可：**第一步，打开遥控器**



第二步，连接接收机





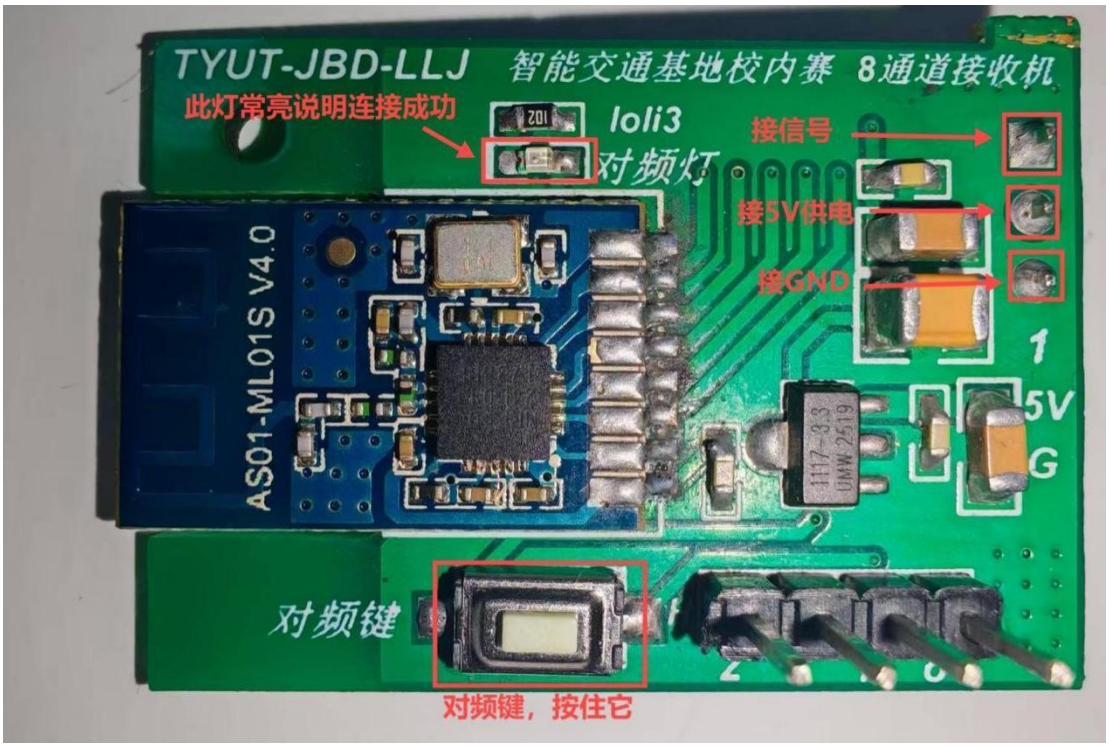






在等待连接期间，拿出接收机，按住接收机上的对频键（接收机上只有一个按键，按住它就行），在按住不松手的情况下给接收机上电，等待接收机上的对频灯闪烁。





接收机示意图

三、如何配置你的舵机或电机

这里的“配置”仅仅指在历程中使用已经配置好的舵机口和电机口，基本不涉及原理，对原理感兴趣的同学可以上网搜索“PWM 原理”。

如何驱动舵机：

舵机的控制只需要给它对应的导线接通 5V，GND 和 PWM 口即可，5V 和 GND 都是通用的，因此我们想要配置舵机时就只需要关注 **PWM 口**的配置即可。在例程中想要配置 PWM 口只需要按照以下步骤进行。

第一步，选取引脚

```
use PWM引脚 A0  A2  A3  A6  A7  B0 eg.驱动舵机  PWM(PWM引脚, 750);
```

第二步，输出控制

对于输出控制，我们需要用到一个函数 PWM_SetDuty()。PWM_SetDuty()这个函数有两个参数，就是说在使用这个函数时，需要往括号里输入两个值，第一个是刚刚自定义的 PWM 通道的名字，第二个是参数叫做占空比，决定了舵机打角的大小，两个参数之间用英文逗号分隔开（**！！！一定要用英文逗号，不要写成中文的！！！**）。

占空比 250 对应了舵机的最小打角，1250 对应了舵机的最大打角，中间占空比与打角大小成正比。

```
PWM(A0, 750);
```

最后正确地插上舵机的三根线，就可以控制舵机的转动了。

电机口：

```
/*控制电机的IO口和PWM口
  IO引脚 B4   D2   C13  B5
  PWM引脚 B6   B7   B8   B9*/
// LF左前轮
#define Motor_LF_IO_GPIO_Port   GPIOB
#define Motor_LF_IO_Pin         GPIO_PIN_4
#define Motor_LF_PWM            PWM_TIM4_CH4_B9

// LB左后轮
#define Motor_LR_IO_GPIO_Port    GPIOD
#define Motor_LR_IO_Pin          GPIO_PIN_2
#define Motor_LR_PWM             PWM_TIM4_CH2_B7

// RF右前轮
#define Motor_RF_IO_GPIO_Port    GPIOC
#define Motor_RF_IO_Pin          GPIO_PIN_13
#define Motor_RF_PWM             PWM_TIM4_CH3_B8

// RB右后轮
#define Motor_RR_IO_GPIO_Port    GPIOB
#define Motor_RR_IO_Pin          GPIO_PIN_5
#define Motor_RR_PWM             PWM_TIM4_CH1_B6
```

这是驱动电机 需要一个 IO 口加一个 PWM 口 IO 控制，电机的驱动函数已经封装好，只需要按照上面的介绍正常使用就好。

摩擦轮电机需要两个 io 口控制，一高电平一低电平为工作模式，两低电平为休息模式，两高电平为燃烧模式，会把驱动模块烧坏！！！！！！！千万不可以这么做。
Motor_Drive(B1, SET, B10, RESET); 第一个参数和第三个参数是你自己选择的 io 口 第二个和第四个参数是设置引脚电平的高低 SET 为高 RESET 为低。